

AI + 数据分析产品设计：讨论总结

一、分析框架：LLM原生能力 vs 调工具实现

LLM原生具备的数据分析能力

- 理解数据含义（字段名、业务背景推断）
- 生成分析代码（写代码 ≠ 运行代码）
- 解读统计结论（相关系数、p值等）
- 给出分析框架（漏斗、同期群、RFM等）
- 从文字描述中提取数据规律

必须调工具才能实现的能力

- 真正执行代码（需沙盒环境）
- 读取真实文件（需文件解析器）
- 连接数据库（需DB connector）
- 生成真实图表（需代码执行后渲染）
- 处理大数据量（需先聚合采样）
- 获取实时数据（需API调用）
- 写回/推送结果（状态改变）

核心判断标准

内容生产 vs 状态改变

- 要的是内容产物（图、文章、报告）→ LLM越来越够用，壁垒在交付形态的可用性
- 要的是现实状态改变（邮件发出去、数据写回去）→ 必须有Agent

二、机会判断：数据分析领域的Cursor

类比逻辑

Cursor做的事：把LLM从”代码建议机器”变成”真正在你项目里工作的协作者”，打通三层断层：

- 1. 上下文断层：LLM视野从代码片段 → 整个代码库
- 2. 执行反馈断层：报错自动回传LLM，自动修复，无需手动复制粘贴
- 3. 产物断层：输出不是文字，而是直接写入文件的真实代码变更

数据分析领域的三个对应断层

- 1. 语义断层：LLM不知道数据仓库里有哪些表、字段什么含义、业务口径是什么
- 2. 执行断层：SQL写完要自己复制到数据库工具跑，结果要自己粘回来让LLM解读
- 3. 交付断层：分析结论要自己整理成报告，推送到钉钉/飞书/邮件

核心护城河

不是LLM，不是Agent框架，而是**业务语义层**——能不能让系统真正理解”这家公司的数据”说的是什么意思。这也是现有text-to-SQL产品大多做到一半就卡住的原因：解决了语言转SQL，没解决”SQL里的字段到底对不对”。

三、目标用户与产品策略

两类用户的核心差异

	数据分析师	业务人员
角色	数据生产者	数据消费者
核心交付	可编辑可执行的SQL/Python	直接可读的结论+图表
信任建立	能看到并修改代码	需要看到数据来源引用
获客路径	开发者社区、口碑	企业销售
付费逻辑	个人订阅	企业合同

建议：先做数据分析师

- 他们能判断输出对不对，给最快反馈
- 他们是企业内部”推广者”，做好了帮你向业务方背书

- 产品错了他们会直接告诉你哪里错
 - Cursor已验证这条路
-

四、面向业务人员：四件额外要做的事

第一件（最难）：可信度证明层

业务人员看到数字不知道怎么来的，必须提供证明链。

三层可追溯设计范式：

结论层（日常查看）

└─ "上周DAU 23.2万，环比下降4.3%"

逻辑层（有疑问时展开）

└─ 数据来源 / 统计口径 / 对比基准 / 排除规则
(用业务语言描述，不是SQL)

原始数据层（深度验证）

└─ 实际执行的SQL + 原始数据样本（前100行）

关键设计细节：

- 每个数字有“身份证”：hover显示来源卡片
- 异常主动标注⚠️，给出可能解释
- 口径冲突显性化：让用户参与选择，而非黑盒决定

核心原则：预防性透明

把“可信度”从事后验证变成事前透明。在给数字的同时同步呈现证明链（哪怕收起来），让用户从第一天就知道“这个产品是透明的”。

隐藏价值：内部背书机制 数据分析师是企业内部“数据公证人”，让他们有能力验证和背书，业务人员最终信任的是人，不是工具。

第二件：指标语义的业务翻译

维护“业务语言 → 数据语言”的翻译词典，每家公司不一样（电商的“活跃”≠游戏的“活跃”）。

第三件：追问交互

报告不只是交付物，而是**对话的起点**——业务人员在飞书/钉钉收到报告，直接在里面追问，系统

实时回答，前后用同一套数据口径。

第四件：主动推送（异常监控）

业务人员不会主动打开工具查数，系统必须主动推送——DAU今天跌了10%，主动告诉他，而不是等他来问。从”对话工具”变成”数据岗哨”。

五、ABI vs 纯对话：产品范式选择

两种范式

ABI范式：自然语言 → BI操作（维度/指标/过滤器）→ SQL → 结果

纯对话范式：自然语言 → SQL → 结果 → 自然语言

为什么BI加AI有结构性天花板

- 1. 数据模型是死的：BI语义层由数据工程师提前手工定义，只能回答被预料到的问题，业务人员的临时起意问题往往超出模型边界
- 2. 商业模式冲突：BI厂商的核心价值是Dashboard，AI做好了用户不需要Dashboard，等于拆自己的核心价值，所以动作天然保守

修正：如果AI能动态理解和扩展语义层（自动扫描表/字段、结合历史查询推断含义），天花板可以被打破——但这正是现有产品没做到的地方。

分角色判断

	ABI范式	纯对话范式
数据分析师	✗ 多了一层抽象，验证负担更重	✓ SQL直接可验证可修改
业务人员	✗ 可追溯是假的，看不懂BI操作	✓ 决策可翻译成业务语言真正可懂

核心洞察：ABI给业务人员的”可追溯”是虚假的——他能看到”维度：获客渠道，指标：DAU_去重登录用户”，但他依然无法判断这些操作对不对，和看不懂SQL是同一个问题，只是形式不同。

纯对话范式可以把SQL背后的决策翻译成业务语言：

“我理解你问的是上周自然周、按获客渠道拆分的去重登录用户数，排除了测试账号。其中‘活跃’定义为当日有登录记录。如果你想换成有下单行为才算活跃，告诉我。”

这才是业务人员真正能读懂、能判断、能修改的可追溯。

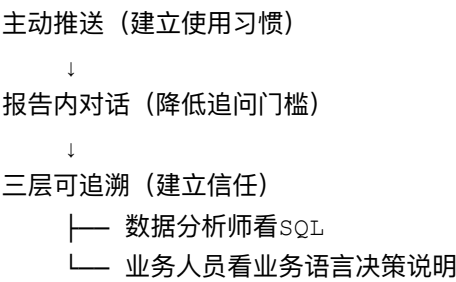
最终产品形态建议

纯对话作为主入口和分析引擎 + BI式可视化作为结果呈现和探索层

- 用对话问问题（明确需求、验证口径）
- 用图表看结果和做探索性分析（拖拽维度、切换时间范围）
- 两者结合，而不是非此即彼

六、正确的产品范式总结

面向两类用户的统一产品逻辑



与竞品的差异化

竞品	做了什么	没做什么
Text-to-SQL工具（Defog/Vanna）	语言转SQL	语义层 + 报告推送
BI Copilot（Tableau Pulse）	对话入口	动态语义层 + 翻译透明化
国内产品（数问/Chat2DB）	对话层	执行闭环 + 报告推送

机会窗口：把语义层 + 执行闭环 + 报告推送三层真正串起来，没有人完整做完这件事。

本文档整理自产品设计讨论，聚焦AI数据分析工具的产品形态、用户分层、可信度设计与竞品分析。